

CHAPTER 14

YouTube 설계

동영상 업로드와 재생 시스템의 핵심 구조를 정리합니다.
업로드 세션, 비동기 트랜스코딩, 메타데이터 관리, CDN 기반 재생,
장애 대응까지 한 번에 훑습니다.

이번 장의 설계 관점

What this chapter focuses on

이번 장의 설계 관점

- 업로드 경로와 재생 경로의 성격이 다르므로 분리해서 생각합니다.
- 대용량 파일은 네트워크 중단에 취약하므로 Resumable Upload를 기본 가정으로 둡니다.
- 업로드 직후 바로 재생되는 것이 아니라, 비동기 트랜스코딩을 거쳐 여러 화질 자산을 만듭니다.

이번 장에서 다루는 범위

- 비디오 업로드, 업로드 상태 추적, 메타데이터 저장
- 트랜스코딩 파이프라인, 썸네일 생성, 재생용 패키징
- CDN을 통한 글로벌 재생과 운영/장애 대응

핵심 질문 : 업로드는 안전하게 받고, 처리 비용은 제어하면서, 재생은 빠르고 안정적으로 제공하는 방법은 무엇인가?

문제 이해 및 범위 정의

Problem understanding & scope

면접관과의 대화 — 확인할 질문

- 지원 기능은 무엇인가요? (업로드, 재생, 메타데이터 조회)
- 클라이언트 종류는 무엇인가요? (Web, Mobile, Smart TV)
- 가장 중요한 사용자 경험은 무엇인가요?
(빠른 시작, 끊김 없는 재생, 업로드 재개)
- 라이브/댓글/추천/검색/광고를 이번 설계에 포함하나요?
- 대략적인 사용자 규모와 업로드 크기는 어느 정도인가요?

5M

DAU (일간 활성 사용자)

300MB

평균 원본 업로드 크기

확정된 제약

- 이번 설계의 중심은 VOD 업로드/재생입니다.
- 댓글, 추천, 광고, 검색 랭킹은 범위 밖으로 둡니다.
- 대용량 업로드와 글로벌 재생이 설계의 핵심 난점입니다.

기능 요구사항 정리

Functional requirements

기능 요구사항

- 사용자는 비디오를 업로드할 수 있어야 합니다.
- 업로드가 중단되어도 이어서 업로드할 수 있어야 합니다.
- 업로드된 비디오는 제목, 설명, 태그, 공개 범위 같은 메타데이터를 가져야 합니다.
- 비디오는 여러 해상도와 비트레이트로 트랜스코딩되어야 합니다.
- 사용자는 비디오 정보를 조회하고 재생할 수 있어야 합니다.
- 시스템은 업로드/처리/재생 상태를 추적할 수 있어야 합니다.

면접에서는 기능 요구사항을 업로드, 처리, 재생 세 단계로 나누어 설명하면 구조가 깔끔해집니다.

비기능 요구사항 정리

Non-functional requirements

품질 속성

- Scalability — 인기 비디오의 폭발적 읽기 트래픽 처리
- Reliability — 업로드 도중 끊겨도 데이터 유실 방지
- Availability — 일부 장애에도 서비스 유지

운영 속성

- Cost efficiency — 저장 비용과 CDN 비용을 통제
- Low latency — 재생 시작 시간과 API 응답 지연 최소화
- Quality of experience — 재생 중 버퍼링이 발생하는 횟수를 줄이고 화질 전환을 부드럽게 유지

대략적 규모 가정

Back-of-the-envelope estimation

5M

DAU

10%

가정: 하루 업로드 사용자 비율

300MB

평균 원본 업로드 크기

업로드 트래픽 추정

5M DAU 중 10%가 하루 1개 업로드한다고 가정하면
하루 약 500K uploads가 발생합니다.
업로드 데이터 유입량은 약 150TB/day 수준입니다.

왜 중요한가?

업로드 트래픽만 봐도 상당하지만, 실제 더 큰 문제는
재생 트래픽입니다. 따라서 설계의 무게중심은 객체 저장소와
CDN으로 이동합니다.

계산이 알려주는 설계 방향

Storage-heavy and bandwidth-heavy system

계산이 알려주는 설계 방향

- 원본 비디오는 DB가 아니라 객체 저장소에 보관해야 합니다.
- 트랜스코딩 자산까지 고려하면 저장량은 원본의 2~4배로 늘어날 수 있습니다.
- 읽기 트래픽이 훨씬 크므로 재생은 CDN 중심으로 설계해야 합니다.

Bandwidth 관점

- 업로드 경로는 쓰기 중심, 재생 경로는 읽기 중심입니다.
- 경로를 분리하면 각 병목을 독립적으로 최적화할 수 있습니다.
- 메타데이터 API와 실제 비디오 전송을 다른 계층으로 나누는 것이 좋습니다.

이 시스템은 저장 공간이 매우 중요하면서 동시에 대역폭이 매우 중요한 시스템입니다.

API 설계 업로드

Upload session, chunk upload, completion

POST /v1/videos

비디오 메타데이터를 먼저 등록하고 video_id, upload_session_id를 발급합니다. 상태는 PENDING 또는 UPLOADING으로 시작합니다.

PUT /v1/uploads/{sessionId}/chunks

chunk payload를 전송합니다. Content-Range, chunk_index, checksum, idempotency key를 함께 보내 재개 가능하게 만듭니다.

GET /v1/uploads/{sessionId}

현재까지 저장된 offset, missing chunks, 세션 만료 시간을 반환합니다. 이어받기용 API입니다.

POST /v1/uploads/{sessionId}/complete

모든 chunk 수신을 검증하고 원본 객체를 확정합니다. 이후 transcode job을 큐에 넣습니다.

API 설계 재생

Metadata lookup and playback URL

재생 관련 API

GET /v1/videos/{videoid}

제목, 길이, 썸네일, 상태, 공개 범위 등 메타데이터 조회

GET /v1/videos/{videoid}/playback

manifest URL(HLS/DASH), 사용 가능한 화질, 자막 정보, DRM 토큰 등 반환

예시 응답 필드

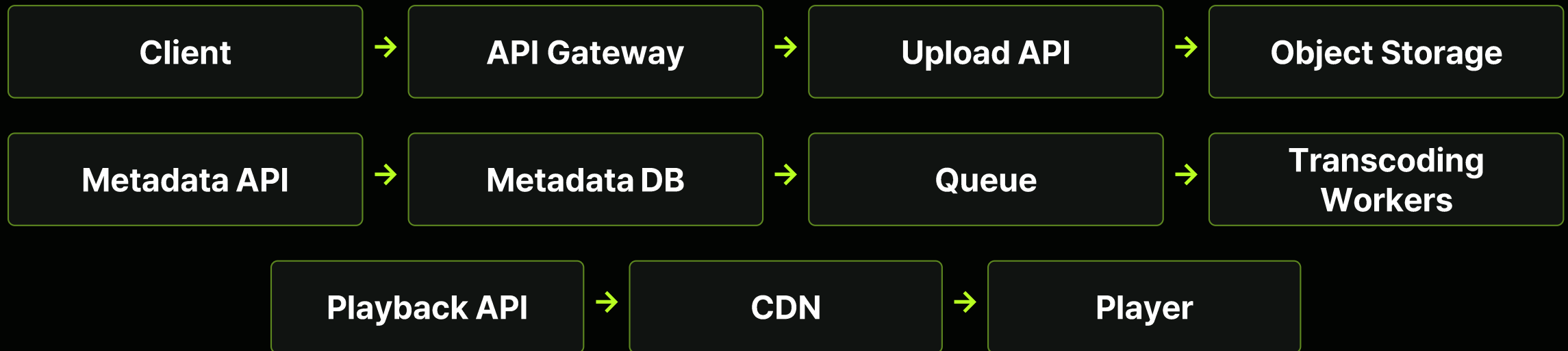
- video_id, title, description, duration, status
- playback_url, thumbnails, available_variants
- captions, signed_token, expiration_time

재생 API의 핵심은 비디오를 직접 내려주는 것이 아니라, 재생에 필요한 메타데이터와 manifest URL을 제공하는 것입니다.

개략적 아키텍처

High-level architecture

개략적 아키텍처



Control plane

사용자 인증, 세션 관리, 메타데이터 조회, 상태 전환 같은 작은 요청을 처리합니다.

Data plane

원본 비디오 업로드와 재생 segment 전송 같은 대용량 데이터 이동을 담당합니다.

업로드 상세 흐름

End-to-end upload flow

1 세션 생성

video_id, session_id, 업로드 정책을 발급합니다.

2 Chunk 업로드

클라이언트가 원본을 chunk로 나누어 전송합니다.

3 중간 상태 기록

offset, chunk bitmap, checksum을 저장합니다.

4 업로드 완료

모든 chunk 수신 후 complete 호출로 마감합니다.

5 원본 확정

세션을 CLOSED로 전환하고 원본 객체를 고정합니다.

6 작업 큐 등록

transcode, thumbnail, moderation 작업을 큐에 넣습니다.

7 트랜스코딩

여러 해상도의 자산과 manifest를 생성합니다.

8 공개

상태를 READY로 바꾸고 재생이 가능해집니다.

업로드 최적화 **Chunk와 Resume**

Chunked upload, retries, idempotency

Chunk upload 장점

- 대용량 파일을 메모리에 한 번에 올리지 않아도 됩니다.
- 병렬 업로드가 가능해 업로드 시간을 줄일 수 있습니다.
- 손실된 일부 chunk만 재전송하면 됩니다.

Resume 처리

- 서버는 마지막 offset 또는 수신된 chunk 목록을 저장합니다.
- 클라이언트는 missing chunk만 다시 전송합니다.
- 각 chunk에 checksum을 두어 무결성을 검증합니다.

Idempotency

- 같은 chunk가 재전송되어도 중복 반영되지 않게 합니다.
- chunk index + session_id 조합으로 중복 여부를 판단합니다.
- complete 요청도 멍등성을 지키도록 만드는 것이 좋습니다.

저장소 설계 원본과 자산

Raw object, transcoded asset, thumbnail

Raw video storage

- 원본 비디오는 수정 불가능한 객체로 저장합니다.
- 보통 파일명은 video_id 기반으로 구성합니다.
- 내구성이 높은 객체 저장소를 사용합니다.

Encoded asset storage

- 240p, 360p, 480p, 720p, 1080p 등 다양한 해상도로 저장
- HLS/DASH segment와 manifest도 함께 저장
- 트랜스코딩 결과는 CDN origin 역할을 합니다.

Thumbnail storage

- 대표 썸네일과 여러 크기의 썸네일 저장
- 자주 조회되므로 캐시 친화적입니다.
- 작지만 조회가 많아 운영상 중요합니다.

면접에서는 “비디오 데이터는 객체 저장소, 메타데이터는 DB”라고 명확히 분리해서 말하는 것이 중요합니다.

트랜스코딩이 필요한 이유

Why transcoding matters

왜 필요한가?

- 업로드된 원본 포맷은 모든 기기에서 바로 재생되지 않을 수 있습니다.
- 네트워크 환경에 따라 적절한 화질이 다릅니다.
- CDN/플레이어 친화적인 segment 포맷으로 바뀌야 합니다.

생성되는 자산

- 여러 해상도와 비트레이트 variant
- 키프레임 기반 썸네일
- HLS .m3u8 또는 DASH .mpd manifest

트랜스코딩은 사용자 경험과 비용 효율을 동시에 좌우하는 단계입니다.

비디오 처리 파이프라인

From uploaded file to playable assets

1. 업로드 완료 & 원본 확정

2. 포맷 검증 / 바이러스 스캔 / 기본 메타 추출

3. 여러 해상도와 비트레이트로 트랜스코딩

4. 썸네일 / keyframe / 자막 처리

5. HLS/DASH packaging 및 manifest 생성

6. 메타데이터 갱신 후 READY 상태로 공개

트랜스코딩 DAG 모델

Workflow processing as a directed acyclic graph

DAG로 표현하는 이유

- 작업 간 선후 관계가 명확합니다.
- 실패한 노드만 재시도할 수 있습니다.
- 병렬화 가능한 작업을 쉽게 찾을 수 있습니다.

예시 노드

- 메타데이터 추출
- 360p / 720p / 1080p 트랜스코딩
- 썸네일 생성, HLS 패키징, 메타데이터 업데이트

예 : 360p와 720p 트랜스코딩은 병렬 실행할 수 있고, manifest 생성은 모든 variant가 준비된 뒤 실행합니다.

DAG Scheduler와 Task 분배

Scheduling and task orchestration

DAG Scheduler 역할

- 작업을 큐에 넣고 의존 관계를 추적합니다.
- 실행 가능한 노드만 worker에 배정합니다.
- 실패 시 retry/backoff/DLQ 정책을 적용합니다.

작업 메타데이터

- task_id, video_id, stage, input_uri
- retry_count, priority, timeout
- dependencies, output_uri, status

이 계층이 있으면 파이프라인이 커져도 제어가 쉽고, moderation/subtitle/fingerprinting도 자연스럽게 붙일 수 있습니다.

Resource Manager와 Worker Pool

Task queue, worker, scaling

Resource Manager

- 작업 종류별 CPU/GPU 자원을 추상화합니다.
- 코덱별로 워커 풀을 나눌 수 있습니다.
- 혼잡 시 우선순위 정책을 적용합니다.

Worker Pool

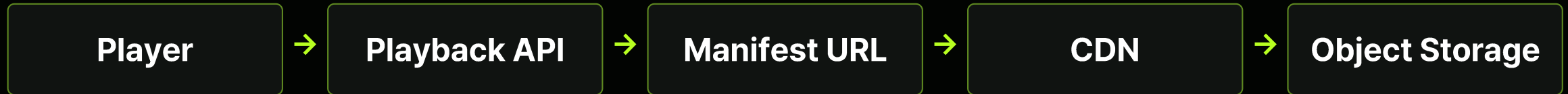
- 작업을 수행하는 워커 집합입니다.
- 역등성을 지키면서 실행해야 합니다.
- 진행률을 주기적으로 저장합니다.

Running Queue

- 길이에 따라 오토 스케일링합니다.
- 긴/짧은 작업을 분리할 수 있습니다.
- DLQ를 통해 영구 실패를 분리합니다.

재생 경로 개요

Manifest, segment, origin, CDN



Manifest 역할

플레이어에게 어떤 화질 variant가 있는지 알려주고, 각 variant의 segment 경로를 제공합니다.

Segment 역할

실제 재생 데이터입니다. 작은 조각 단위로 나누어 전송하므로 화질 전환과 캐시가 쉬워집니다.

CDN 기반 재생 경로

Cache hit, cache miss, origin fetch

CDN 기반 재생 경로

- 플레이어는 playback API로 manifest URL을 받습니다.
- 실제 미디어 segment는 가능하면 CDN에서 내려받습니다.
- cache miss일 때만 origin storage로 갑니다.

왜 중요한가?

- 읽기 트래픽 대부분을 origin 밖으로 밀어낼 수 있습니다.
- 사용자와 가까운 edge에서 응답하므로 지연이 줄어듭니다.
- 인기 비디오가 몰려도 저장소와 애플리케이션 서버를 보호할 수 있습니다.

CDN 비용 최적화

Popular videos vs long-tail videos

Popular video 전략

- 조회수가 빠르게 증가하는 비디오는 edge cache를 빠르게 채웁니다.
- 첫 segment와 manifest는 미리 warm-up하면 startup latency를 줄일 수 있습니다.
- 다중 CDN 전략으로 지역별 비용/품질을 조절할 수 있습니다.

Long-tail video 전략

- 드물게 재생되는 비디오는 모든 edge에 미리 배포하지 않습니다.
- 요청이 생길 때 원본에서 가져와 lazy caching합니다.
- 오래된 variant는 저렴한 storage tier로 이동할 수 있습니다.

CDN은 성능만이 아니라 비용 최적화의 핵심입니다.

데이터 모델 기본 스키마

Core metadata schema

테이블	Key fields	설명
videos	video_id, owner_id, title, status, duration, privacy	비디오 메타데이터의 중심
upload_sessions	session_id, video_id, offset, state, expires_at	resumable upload 상태 추적
video_assets	video_id, variant, codec, bitrate, url, size	트랜스코딩 자산과 manifest/thumbnaill 위치

메타데이터는 크기가 작고 강한 일관성이 필요한 반면, 비디오 데이터는 매우 크고 수정이 거의 불가능하다는 점을 구분해야 합니다.

DB, Cache, Sharding

Metadata storage strategy

Primary key

- video_id, session_id 기준 단건 조회가 많습니다.
- 짧은 응답시간을 위해 PK 조회 경로를 최적화합니다.

Secondary index

- owner_id + created_at
- status + created_at
- 필요 시 privacy + owner_id

Sharding

- video_id hash 기준 분산이 일반적입니다.
- Creator 목록 조회는 secondary index나 별도 materialized view로 보완합니다.

재생 트래픽과 캐시 계층

Layered caching strategy

Metadata cache

Video metadata 단건 조회를 빠르게 처리합니다.

Thumbnail cache

썸네일은 매우 자주 조회되므로 캐시 효율이 높습니다.

Manifest cache

manifest는 작고 재사용성이 높아 캐시 적중률이 좋습니다.

Segment cache

실제 비용 절감과 지연 개선의 핵심 계층입니다.

Hot object는 여러 계층의 캐시를 타지만, Cold data는 결국 원본 저장소에서 읽히므로 Cache policy가 중요합니다.

속도 최적화 전략

Latency and throughput improvements

속도 최적화

- Parallel upload lanes
- Resume from last valid offset
- First segment prefetch

스토리지/서빙 최적화

- Frequently viewed content pinning
- Origin shield, persistent connections
- Adaptive chunk/segment size

처리 파이프라인 최적화

- Priority-based scheduling
- GPU transcoding for hot uploads
- Batch thumbnail extraction

신뢰성과 장애 대응

Upload path reliability

업로드 중 장애

- 네트워크가 끊겨도 세션 상태를 조회해 재개합니다.
- Chunk 단위 checksum으로 손상 여부를 검사합니다.
- 세션에는 TTL을 두고 오래된 세션을 정리합니다.

백엔드 장애

- API 서버는 stateless하게 두고 수평 확장합니다.
- 메타데이터 DB는 복제와 failover를 사용합니다.
- 원본이 안전하게 저장되기 전에는 COMPLETE 처리하지 않습니다.

재생 경로 장애 대응

Playback reliability and fallback

재생 경로 장애

- CDN 장애 시 다른 CDN 또는 origin fallback을 고려합니다.
- Manifest TTL을 짧게 두면 경로 전환이 쉬워집니다.
- 특정 variant가 실패해도 더 낮은 화질로 재생을 이어갈 수 있습니다.

트랜스코딩 실패

- Worker retry, backoff, DLQ를 둡니다.
- 일부 variant만 실패한 경우 부분 실패 상태를 추적합니다.
- Graceful degradation으로 핵심 재생을 우선 보장합니다.

보안, 권한, 콘텐츠 보호

Security and moderation

Authentication

사용자 인증은 OAuth/JWT 등을 사용합니다.

Authorization

비공개/일부 공개/연령 제한 정책을 반영합니다.

Signed URLs

업로드와 재생 경로는 짧은 TTL의 서명 URL을 사용할 수 있습니다.

Rate limit

악성 업로드, 크롤링, abuse를 막기 위해 제한을 둡니다.

콘텐츠 검수, 바이러스 스캔, 저작권 또한 실제 서비스에서 매우 중요합니다.

운영 지표와 관측성

Monitoring, metrics, tracing

운영 지표

- Upload success rate, average upload duration
- Transcode queue length, transcode latency
- CDN hit ratio, playback startup time

가시성

- video_id와 session_id를 기준으로 E2E trace 연결
- Worker backlog 급증, 특정 리전 오류 증가 시 alert 발생
- 로그, 메트릭, 추적을 함께 봐야 원인 분석이 가능합니다.

트레이드오프와 마무리

Trade-offs and recap

주요 트레이드오프

- Proxy upload vs Direct-to-storage upload
- Immediate transcoding vs Delayed transcoding
- 강한 일관성 vs 최종 일관성

면접 마무리에서 강조할 점

- 업로드와 재생 경로를 분리했다는 점
- 객체 저장소 + 메타데이터 DB 분리
- 비동기 트랜스코딩 + CDN 기반 재생

핵심 요약

End-to-end summary

1. Upload session

대용량 업로드는 세션과 chunk 단위 전송으로 안정성을 확보합니다.

2. Object storage

원본과 트랜스코딩 자산은 객체 저장소에 둡니다.

3. Metadata DB

작은 메타데이터와 상태는 DB/캐시에 저장합니다.

4. Async processing

트랜스코딩은 큐와 워커가 비동기로 처리합니다.

5. CDN playback

글로벌 재생은 CDN, manifest, segment 구조로 풀어냅니다.

6. Reliability

재시도, fallback, observability가 운영 품질을 결정합니다.